Universidad Simón Bolívar

Departamento de Computación y Tecnología de la Información

CI-4835 Redes de Computadoras I

Profesor: Ing. Samuel Sánchez

PROYECTO 1
CHAT MULTIUSUARIO ESTILO IRC CON PROTOCOLO DE TEXTO

Estudiantes:

Moisés Blanco 21-10074

Sebastián Moglia 18-10254

Valle de Sartenejas, octubre del 2026

1. Objetivo y arquitectura

Se implementó un sistema de chat multiusuario en C++17 utilizando directamente sockets TCP POSIX. El proyecto comprende un servidor de chat con apodos, canales y mensajes privados; un buzón persistente para usuarios desconectados; una puerta HTTP/1.1 de solo lectura; y un cliente de terminal.

El protocolo de aplicación emplea texto UTF-8, comandos delimitados por CRLF, respuestas numéricas y eventos asíncronos. Los archivos README.md y PROTOCOLO.md documentan su ejecución y su gramática.

2. Modelo de concurrencia y estado compartido

El servidor utiliza el modelo de un hilo por conexión de chat. El hilo principal acepta conexiones en el puerto indicado por --chat y crea un std::thread para cada cliente. Una tarea independiente atiende el puerto HTTP y crea hilos para sus consultas.

Elegimos este modelo porque permite procesar conexiones de forma independiente y simplifica la implementación de operaciones bloqueantes de recepción con la API POSIX, sin incorporar bibliotecas de red externas.

El estado global contiene los usuarios registrados y conectados, los canales, sus miembros y sus historiales. Se protege mediante std::mutex y std::lock_guard para evitar modificaciones concurrentes inconsistentes.

Para los envíos del chat se utiliza send() con MSG_DONTWAIT y MSG_NOSIGNAL. Si un cliente deja de recibir y el envío no puede completarse, se cierra su conexión para evitar que detenga a los demás. Las consultas HTTP tienen límites de tamaño y tiempos de espera.

Por su parte, el cliente emplea poll() sobre el teclado y el socket, por lo que puede mostrar eventos entrantes mientras el usuario escribe.

3. Delimitación de mensajes TCP y robustez

TCP entrega un flujo de bytes, por lo que una llamada a recv() no corresponde necesariamente a un comando completo. Por ello, el servidor mantiene un búfer por conexión y recorre los bytes recibidos hasta detectar la secuencia CRLF (\r\n).

Cuando completa una línea, la extrae para interpretarla. Si llegan varias líneas juntas, procesa cada una por separado; si una línea llega dividida en diferentes lecturas, conserva los bytes parciales hasta completarla. También contempla que los caracteres CR y LF lleguen en lecturas diferentes.

Cada línea enviada por el cliente tiene un límite de 512 bytes, incluidos los dos bytes de CRLF. Cuando se supera ese límite, el servidor responde con el código 502 y descarta el contenido hasta encontrar el siguiente CRLF, sin perder los comandos posteriores.

Antes de interpretar los comandos también se valida la codificación UTF-8, los nombres y los argumentos recibidos. Los comandos se aceptan sin distinguir entre mayúsculas y

minúsculas. El cliente reconstruye de forma similar las respuestas recibidas, pero no aplica el límite de 512 bytes a las líneas enviadas por el servidor.

4. Protocolo, buzones y puerta HTTP

El registro se realiza mediante NICK. Antes de registrarse solamente se admiten NICK y QUIT. Los comandos JOIN, PART, MSG, PRIV, CHANS y WHO permiten administrar la comunicación entre los usuarios, utilizando respuestas numéricas y eventos asíncronos. Los mensajes enviados a canales se transmiten mediante el evento 300 y los mensajes privados recibidos en línea mediante el evento 301.

Cuando el destinatario de PRIV es un usuario conocido que se encuentra desconectado, el servidor guarda el mensaje en disco como un archivo .eml con formato RFC 5322. Cada archivo contiene las cabeceras From, To, Date, Subject y Message-ID, seguidas de una línea vacía y el cuerpo del mensaje. Las líneas terminan en CRLF.

Los archivos tienen nombres únicos y permanecen almacenados después de reiniciar el servidor. Los comandos STAT, LIST y RETR permiten consultar el buzón, mientras que DELE marca los mensajes para su eliminación. Los borrados se aplican al ejecutar QUIT; si la conexión termina inesperadamente, los mensajes se conservan.

La puerta HTTP/1.1 implementa las rutas GET /usuarios, GET /canales y GET /historial/<canal>. El servidor acumula la petición hasta encontrar CRLF y genera respuestas con los códigos 200, 400, 404 y 405. Todas incluyen las cabeceras Content-Type, Content-Length y Connection: close. Cuando el método no está permitido, también se incluye Allow: GET. Después de responder, la conexión HTTP se cierra.

5. Verificación y pruebas

El sistema se compiló desde la raíz del repositorio mediante make y se comprobó el funcionamiento de make clean. Se realizaron pruebas con varios clientes conectados para verificar el registro, la creación de canales, el intercambio de mensajes, el almacenamiento y recuperación de mensajes privados, así como el borrado diferido del buzón.

También se comprobaron las tres rutas HTTP disponibles y las respuestas correspondientes a peticiones correctas, mal formadas, rutas inexistentes y métodos no permitidos.

Mediante scripts de Python se probaron comandos TCP enviados juntos y fragmentados en varias partes. Se verificó la aceptación de líneas de exactamente 512 bytes, el rechazo de líneas de 513 bytes y la recuperación del servidor después de recibir una línea de 10 MiB.

Además, se completó una prueba de 200 conexiones simultáneas. Para evaluar el comportamiento ante clientes lentos, se enviaron 3000 mensajes mientras uno de los receptores dejaba de leer. Durante esa prueba, otro cliente pudo ejecutar CHANS y obtuvo una respuesta en 0,041 segundos.

Finalmente, se generó el archivo captura.pcapng mediante TShark, registrando una sesión que incluye NICK, JOIN, MSG, un PRIV dirigido a un usuario desconectado, RETR y una consulta HTTP.

Estas pruebas permiten comprobar el funcionamiento de las características principales del sistema, aunque no garantizan la ausencia de errores en todos los escenarios posibles.

6. Limitaciones conocidas

El modelo de concurrencia utilizado crea un hilo por cliente, por lo que el consumo de recursos aumenta con la cantidad de conexiones simultáneas. No se implementó un grupo fijo de trabajadores.

Algunas operaciones sobre los archivos del buzón se realizan mientras se mantiene adquirido el mutex global. Esto significa que una operación de disco lenta podría retrasar temporalmente la atención de otras conexiones.

El historial de los canales se conserva únicamente en memoria y se pierde al reiniciar el servidor. En cambio, los mensajes almacenados en los buzones sí permanecen disponibles después de un reinicio.

No se implementaron conexiones HTTP persistentes ni las demás bonificaciones opcionales del proyecto. Asimismo, cuando un cliente deja de leer y no es posible completar el envío de información, el servidor puede desconectarlo para evitar que afecte al resto de los usuarios.

7. Distribución del trabajo y uso de inteligencia artificial

El desarrollo del proyecto fue realizado por ambos integrantes. Moisés Blanco se encargó principalmente de la implementación inicial del sistema, mientras que Sebastián Moglia participó en el desarrollo de otras funcionalidades, así como en la revisión, corrección, integración y realización de pruebas. Ambos colaboraron en la resolución de problemas y en las mejoras necesarias para cumplir con los requisitos del proyecto.

Durante el desarrollo se utilizaron ChatGPT y DeepSeek como herramientas de apoyo para resolver dudas, comprender el funcionamiento de determinadas partes del código y consultar posibles formas de implementar o corregir algunas funcionalidades. También se utilizaron para proponer y generar modificaciones en el código, que posteriormente fueron integradas y verificadas por el equipo.

Adicionalmente, se utilizó ChatGPT para generar scripts de prueba en Python, con los cuales se evaluó el comportamiento del servidor ante comandos TCP fragmentados, varios comandos enviados juntos, líneas de diferentes tamaños, 200 conexiones simultáneas y clientes lentos.

Las pruebas fueron ejecutadas en GitHub Codespaces y sus resultados fueron revisados por el equipo.